

Домашнее задание

Интеграция ClickHouse с Apache Kafka

Курс	ClickHouse
Тема	Интеграция с Apache Kafka
Среда выполнения	Локально на macOS с использованием Docker Desktop
Компоненты	Apache Kafka, ClickHouse Server, Docker Compose, Python consumer
Результат	Настроены основной пайплайн и два дополнительных задания со звездочкой

Цель работы: изучить интеграцию ClickHouse с Kafka, настроить пайплайн записи и чтения данных, а также проверить корректность передачи данных между Kafka и ClickHouse.

1. Цель и задачи работы

Цель работы — изучить интеграцию ClickHouse с Apache Kafka и настроить пайплайн для записи и чтения данных.

- установить и запустить Apache Kafka;
- установить и запустить ClickHouse;
- записать данные в Kafka;
- настроить чтение Kafka topic через ClickHouse Kafka Engine;
- переместить данные в таблицу MergeTree через Materialized View;
- проверить корректность чтения данных из Kafka в ClickHouse;
- выполнить дополнительные задания: запись в Kafka через ClickHouse Kafka Engine и пайплайн без Kafka Engine.

2. Используемая инфраструктура

Работа выполнена локально на macOS. Для изоляции сервисов использован Docker Desktop и Docker Compose. Kafka и ClickHouse запущены в отдельных Docker-контейнерах в одной Docker-сети.

Компонент	Назначение	Контейнер / образ
Apache Kafka	Брокер сообщений и хранение topic	kafka / apache/kafka
ClickHouse Server	Хранение и обработка данных	clickhouse / clickhouse/clickhouse-server
Python consumer	Внешнее чтение Kafka без Kafka Engine	python:3.12-slim

3. Основной пайплайн Kafka → ClickHouse

Основная схема пайплайна:

```
Kafka topic events
    ↓
ClickHouse table hw.kafka_events ENGINE = Kafka
    ↓ Materialized View hw.events_mv
ClickHouse table hw.events_mt ENGINE = MergeTree
```

3.1. Создание Kafka topic

Для передачи событий был создан Kafka topic events с одной партицией и replication factor 1.

```
docker exec -it kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka:9092 \
  --create \
  --topic events \
  --partitions 1 \
  --replication-factor 1
```

```

~/otus/clickhouse-kafka-hw (2.046s)
docker exec -it kafka /opt/kafka/bin/kafka-topics.sh \
--bootstrap-server kafka:9092 \
--create \
--topic events \
--partitions 1 \
--replication-factor 1
Created topic events.

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug kaf
ka
Learn more at https://docs.docker.com/go/debug-cli/

```

Рисунок 1. Создание Kafka topic events.

```

~/otus/clickhouse-kafka-hw (1.536s)
docker exec -it kafka /opt/kafka/bin/kafka-topics.sh \
--bootstrap-server kafka:9092 \
--list
events

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug kaf
ka
Learn more at https://docs.docker.com/go/debug-cli/

```

Рисунок 2. Проверка списка topic: создан topic events.

3.2. Таблицы ClickHouse

В ClickHouse была создана база hw и три объекта: целевая таблица MergeTree, Kafka Engine таблица и Materialized View.

```

CREATE DATABASE IF NOT EXISTS hw;

CREATE TABLE IF NOT EXISTS hw.events_mt
(
    event_id UInt64,
    user_id UInt64,
    event_type String,
    amount Float64,
    created_at DateTime
)
ENGINE = MergeTree
ORDER BY (created_at, event_id);

CREATE TABLE IF NOT EXISTS hw.kafka_events
(
    event_id UInt64,
    user_id UInt64,
    event_type String,
    amount Float64,
    created_at DateTime
)
ENGINE = Kafka
SETTINGS
    kafka_broker_list = 'kafka:9092',
    kafka_topic_list = 'events',
    kafka_group_name = 'clickhouse_events_group_v1',
    kafka_format = 'JSONEachRow',
    kafka_num_consumers = 1;

CREATE MATERIALIZED VIEW IF NOT EXISTS hw.events_mv
TO hw.events_mt
AS

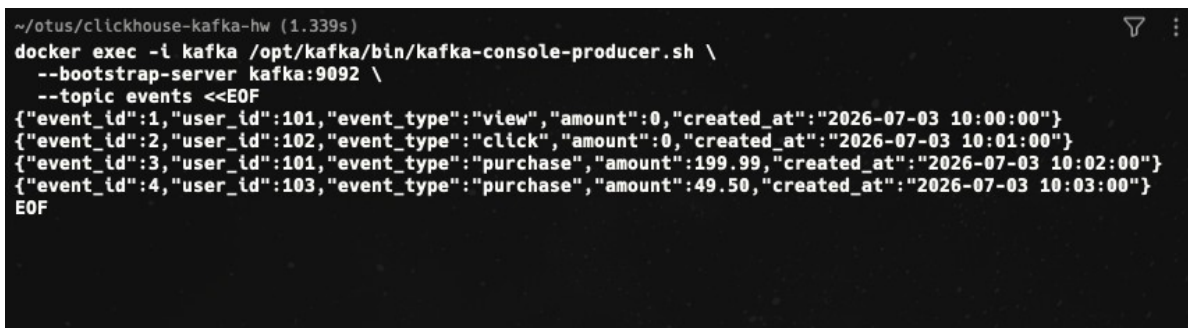
```

```
SELECT event_id, user_id, event_type, amount, created_at
FROM hw.kafka_events;
```

3.3. Запись данных в Kafka

Тестовые сообщения были записаны в Kafka topic events через kafka-console-producer. Формат сообщений — JSONEachRow, соответствующий схеме таблиц ClickHouse.

```
docker exec -i kafka /opt/kafka/bin/kafka-console-producer.sh \
  --bootstrap-server kafka:9092 \
  --topic events <<EOF
{"event_id":1,"user_id":101,"event_type":"view","amount":0,"created_at":"2026-07-03 10:00:00"}
{"event_id":2,"user_id":102,"event_type":"click","amount":0,"created_at":"2026-07-03 10:01:00"}
{"event_id":3,"user_id":101,"event_type":"purchase","amount":199.99,"created_at":"2026-07-03
10:02:00"}
{"event_id":4,"user_id":103,"event_type":"purchase","amount":49.50,"created_at":"2026-07-03
10:03:00"}
EOF
```



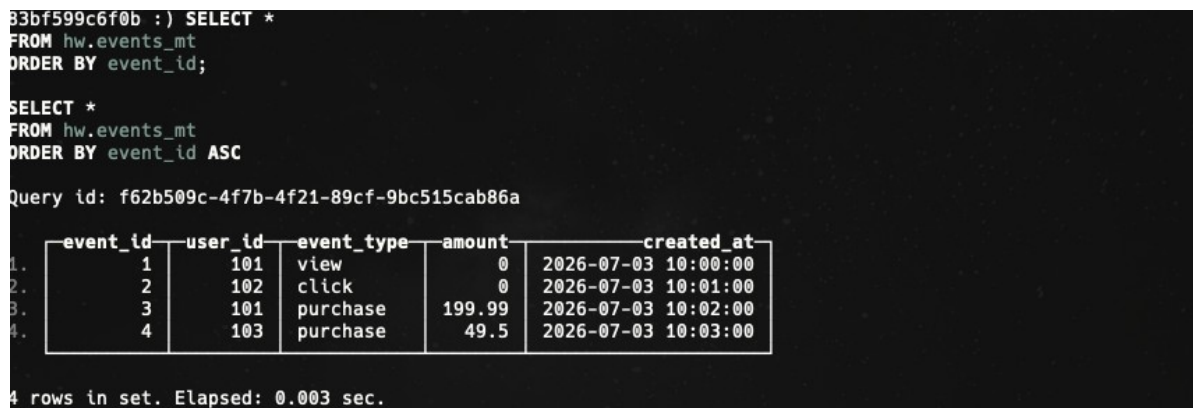
```
~/otus/clickhouse-kafka-hw (1.339s)
docker exec -i kafka /opt/kafka/bin/kafka-console-producer.sh \
  --bootstrap-server kafka:9092 \
  --topic events <<EOF
{"event_id":1,"user_id":101,"event_type":"view","amount":0,"created_at":"2026-07-03 10:00:00"}
{"event_id":2,"user_id":102,"event_type":"click","amount":0,"created_at":"2026-07-03 10:01:00"}
{"event_id":3,"user_id":101,"event_type":"purchase","amount":199.99,"created_at":"2026-07-03 10:02:00"}
{"event_id":4,"user_id":103,"event_type":"purchase","amount":49.50,"created_at":"2026-07-03 10:03:00"}
EOF
```

Рисунок 3. Запись тестовых JSON-сообщений в Kafka topic events.

3.4. Проверка результата в MergeTree

После записи сообщений в Kafka данные были автоматически прочитаны таблицей hw.kafka_events и перенесены Materialized View в таблицу hw.events_mt.

```
SELECT *
FROM hw.events_mt
ORDER BY event_id;
```



```
33bf599c6f0b :) SELECT *
FROM hw.events_mt
ORDER BY event_id;

SELECT *
FROM hw.events_mt
ORDER BY event_id ASC

Query id: f62b509c-4f7b-4f21-89cf-9bc515cab86a

+----+-----+-----+-----+-----+
| event_id | user_id | event_type | amount | created_at |
+----+-----+-----+-----+-----+
1. | 1 | 101 | view | 0 | 2026-07-03 10:00:00 |
2. | 2 | 102 | click | 0 | 2026-07-03 10:01:00 |
3. | 3 | 101 | purchase | 199.99 | 2026-07-03 10:02:00 |
4. | 4 | 103 | purchase | 49.5 | 2026-07-03 10:03:00 |
+----+-----+-----+-----+-----+

4 rows in set. Elapsed: 0.003 sec.
```

Рисунок 4. Проверка загрузки данных в таблицу hw.events_mt.

Результат проверки: в таблице hw.events_mt получено 4 строки. Значит основной пайплайн Kafka → Kafka Engine → Materialized View → MergeTree работает корректно.

4. Дополнительное задание 1: запись в Kafka с помощью ClickHouse Kafka Engine

Для записи данных из ClickHouse в Kafka была создана отдельная таблица hw.kafka_events_producer с движком Kafka. В эту таблицу был выполнен INSERT, после чего сообщения появились в topic events_from_clickhouse.

```
CREATE TABLE IF NOT EXISTS hw.kafka_events_producer
(
    event_id UInt64,
    user_id UInt64,
    event_type String,
    amount Float64,
    created_at DateTime
)
ENGINE = Kafka
SETTINGS
    kafka_broker_list = 'kafka:9092',
    kafka_topic_list = 'events_from_clickhouse',
    kafka_group_name = 'clickhouse_producer_group',
    kafka_format = 'JSONEachRow';

INSERT INTO hw.kafka_events_producer
VALUES
    (101, 1001, 'clickhouse_insert_view', 0, now()),
    (102, 1002, 'clickhouse_insert_click', 0, now()),
    (103, 1003, 'clickhouse_insert_purchase', 500.75, now());
```



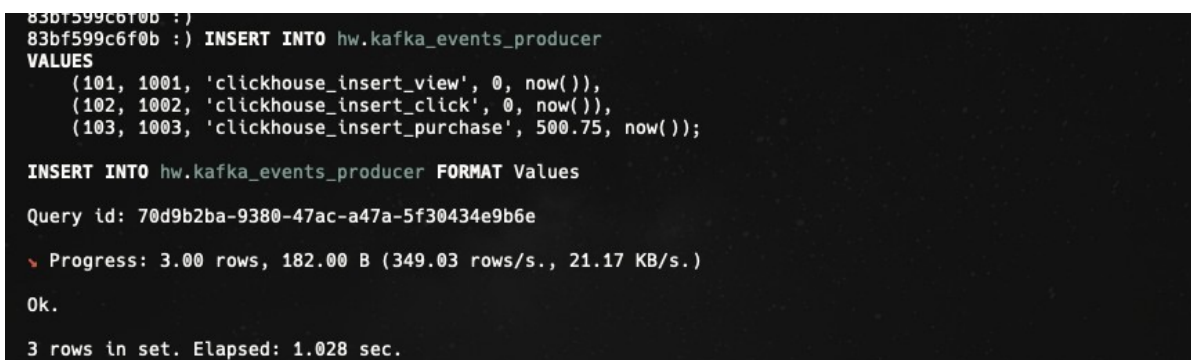
The screenshot shows a terminal window with the following content:

```
~/otus/clickhouse-kafka-hw (1.186s)
docker exec -it kafka /opt/kafka/bin/kafka-topics.sh \
--bootstrap-server kafka:9092 \
--list

__consumer_offsets
events
events_from_clickhouse

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug kaf
ka
Learn more at https://docs.docker.com/go/debug-cli/
```

Рисунок 5. Topic events_from_clickhouse для записи из ClickHouse в Kafka.



The screenshot shows a terminal window with the following content:

```
83bf599c6f0b :)
83bf599c6f0b :) INSERT INTO hw.kafka_events_producer
VALUES
    (101, 1001, 'clickhouse_insert_view', 0, now()),
    (102, 1002, 'clickhouse_insert_click', 0, now()),
    (103, 1003, 'clickhouse_insert_purchase', 500.75, now());

INSERT INTO hw.kafka_events_producer FORMAT Values

Query id: 70d9b2ba-9380-47ac-a47a-5f30434e9b6e

Progress: 3.00 rows, 182.00 B (349.03 rows/s., 21.17 KB/s.)

Ok.

3 rows in set. Elapsed: 1.028 sec.
```

Рисунок 6. INSERT в таблицу hw.kafka_events_producer с ENGINE = Kafka.

Корректность записи была проверена через kafka-console-consumer с чтением topic events_from_clickhouse с начала.

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh \
--bootstrap-server kafka:9092 \
```

```
--topic events_from_clickhouse \  
--from-beginning \  
--timeout-ms 5000
```

```
~/otus/clickhouse-kafka-hw (6.462s)
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh \  
  --bootstrap-server kafka:9092 \  
  --topic events_from_clickhouse \  
  --from-beginning \  
  --timeout-ms 5000

{"event_id":"101","user_id":"1001","event_type":"clickhouse_insert_view","amount":0,"created_at":"2026-07-03 12:13:51"}

{"event_id":"102","user_id":"1002","event_type":"clickhouse_insert_click","amount":0,"created_at":"2026-07-03 12:13:51"}

{"event_id":"103","user_id":"1003","event_type":"clickhouse_insert_purchase","amount":500.75,"created_at":"2026-07-03 12:13:51"}

[2026-07-03 12:14:10,463] ERROR Error processing message, terminating consumer process: (kafka.tools.ConsoleConsumer$)
org.apache.kafka.common.errors.TimeoutException
Processed a total of 3 messages

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug kafka
Learn more at https://docs.docker.com/go/debug-cli/
```

Рисунок 7. Проверка сообщений, записанных ClickHouse в Kafka.

Результат проверки: consumer прочитал 3 сообщения, созданные INSERT-запросом из ClickHouse. Следовательно, запись данных в Kafka через ClickHouse Kafka Engine выполнена успешно.

5. Дополнительное задание 2: пайплайн без Kafka Engine

Во втором дополнительном задании был построен пайплайн без использования ClickHouse Kafka Engine. Чтение из Kafka выполнялось внешним Python consumer, а запись осуществлялась в обычную таблицу MergeTree.

```
Kafka topic events_no_engine
      ↓
External Python consumer
      ↓
ClickHouse table hw.events_no_engine_mt ENGINE = MergeTree
```

5.1. Создание целевой таблицы

```
CREATE TABLE IF NOT EXISTS hw.events_no_engine_mt
(
  event_id UInt64,
  user_id UInt64,
  event_type String,
  amount Float64,
  created_at DateTime
)
ENGINE = MergeTree
ORDER BY (created_at, event_id);
```

5.2. Загрузка через внешний consumer

Внешний Python consumer считал сообщения из Kafka topic events_no_engine и загрузил их в таблицу hw.events_no_engine_mt. В этом варианте ClickHouse не использовал таблицу с ENGINE = Kafka.

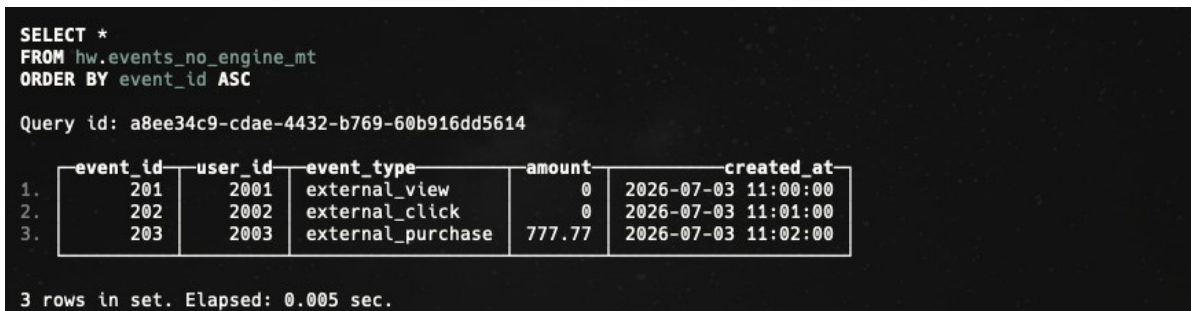
```
docker run --rm \
  --network clickhouse-kafka-hw_default \
  python:3.12-slim \
  python consumer.py
```

Логика consumer:

- # 1. читать сообщения из Kafka topic events_no_engine;
- # 2. сохранять / передавать данные в формате JSONEachRow;
- # 3. вставлять данные в ClickHouse MergeTree таблицу hw.events_no_engine_mt.

5.3. Проверка результата

```
SELECT *
FROM hw.events_no_engine_mt
ORDER BY event_id;
```



The screenshot shows a terminal window with a dark background. It displays the execution of a SQL query in ClickHouse. The query is: `SELECT * FROM hw.events_no_engine_mt ORDER BY event_id ASC`. Below the query, it shows the query ID: `Query id: a8ee34c9-cdae-4432-b769-60b916dd5614`. The result is a table with 5 columns: `event_id`, `user_id`, `event_type`, `amount`, and `created_at`. There are 3 rows of data. At the bottom, it says `3 rows in set. Elapsed: 0.005 sec.`

	event_id	user_id	event_type	amount	created_at
1.	201	2001	external_view	0	2026-07-03 11:00:00
2.	202	2002	external_click	0	2026-07-03 11:01:00
3.	203	2003	external_purchase	777.77	2026-07-03 11:02:00

Рисунок 8. Пайплайн без Kafka Engine: данные загружены во внешнюю MergeTree-таблицу.

*Результат проверки: в таблице hw.events_no_engine_mt получено 3 строки.
Пайплайн Kafka → внешний consumer → MergeTree выполнен без использования
ClickHouse Kafka Engine.*

6. Итоговая проверка выполнения требований

Требование	Что сделано	Статус
Установить Apache Kafka	Kafka запущена в Docker-контейнере kafka.	Выполнено
Установить ClickHouse	ClickHouse Server запущен в Docker-контейнере clickhouse.	Выполнено
Записать данные в Kafka	JSON-события записаны в topic events.	Выполнено
Настроить Kafka Engine	Создана таблица hw.kafka_events с ENGINE = Kafka.	Выполнено
Переложить данные в MergeTree	Создана Materialized View hw.events_mv → hw.events_mt.	Выполнено
Проверить чтение данных	SELECT из hw.events_mt вернул 4 строки.	Выполнено
Звездочка: запись в Kafka через Kafka Engine	INSERT в hw.kafka_events_producer создал 3 сообщения в Kafka.	Выполнено
Звездочка: пайплайн без Kafka Engine	Данные загружены внешним Python consumer в hw.events_no_engine_mt.	Выполнено

7. Вывод

В ходе работы был развернут локальный стенд на Docker Desktop, включающий Apache Kafka и ClickHouse. Были созданы Kafka topics, таблицы ClickHouse с движками Kafka и MergeTree, а также Materialized View для автоматической загрузки данных из Kafka в ClickHouse.

Основной пайплайн Kafka → ClickHouse Kafka Engine → Materialized View → MergeTree успешно проверен: в целевой таблице hw.events_mt появились все отправленные события.

Дополнительно выполнены задания со звездочкой: ClickHouse использован как producer для записи сообщений в Kafka через Kafka Engine, а также реализован альтернативный пайплайн без Kafka Engine с использованием внешнего Python consumer.